

AI 性能は「単体モデル」から「判断と編成」へ

Jev の分類器 / Sakana AI のルーティング / OpenAI のモデル階層から見る次の性能ジャンプ

30 秒で要約

次の AI 性能向上は、単体モデルをさらに巨大化することだけではない。
「状態を高速に判断する分類器」と「最適なモデル・エージェントを選ぶルータ」を組み合わせること
で、同じ計算資源からより高い性能を引き出せる可能性がある。

Jev

生成を捨て、state と命題から typed probabilistic decision を返す「System One Model」。

役割:

- ・意味論的な比較
- ・確率付き判断
- ・低遅延の分岐

Sakana AI

単体モデルを直接強くするだけでなく、Conductor / Fugu が複数モデル・エージェントをタスクごとに編成。

役割:

- ・モデル選択
- ・分業
- ・検証パイプライン

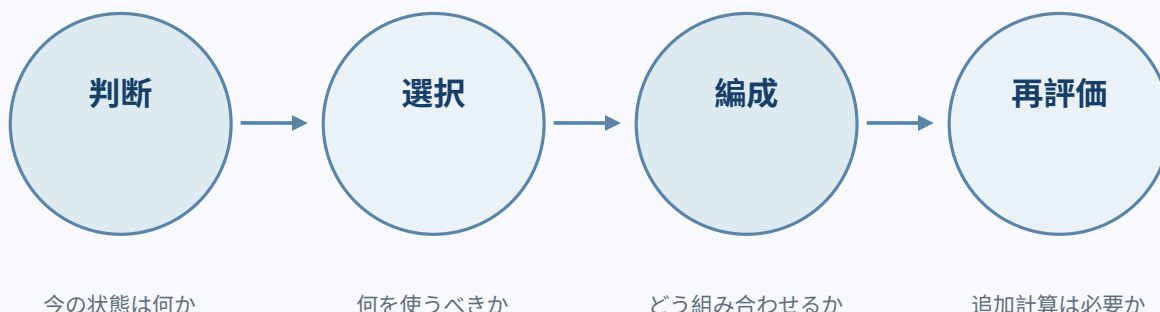
OpenAI

Luna / Terra / Sol / Astra と推論強度を階層化。速度・コスト・能力の意味が分かれている。

可能性:

- ・最適ルーティング
- ・動的 compute 配分

見えてくる共通テーマ



本レポートの結論（先取り）

OpenAI のように役割の異なるモデル階層を持つ企業は、Jev 的な高速分類と Sakana AI 的な学習済みルーティングを取り込んだ場合、「新モデルを作らずに」価格性能比をもう一段ジャンプさせる余地が大きい。

1. Jev と Sakana AI - 「性能の出し方」が違う

生成器を強くする以外に、判断と編成そのものを学習対象にする。

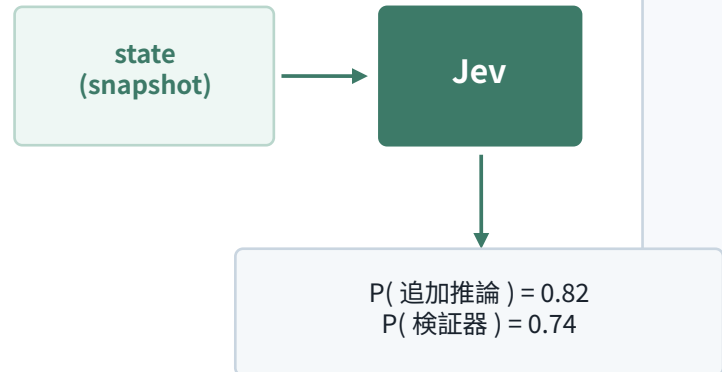
Jev：意味論的・確率的な「比較器」

TypeSafe は Jev を「unstructured state in, typed probabilistic decisions out」と説明。文字列生成ではなく、現在状態と命題の関係を確率として返す。RLCD で calibrated decision を学習し、並列 sampler で多数の判断を一度に処理する。

低遅延

低コスト

確率出力



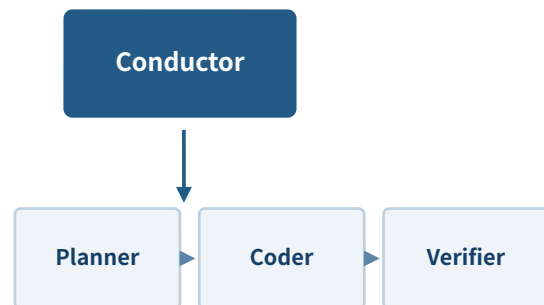
Sakana AI：解くモデルより「編成するモデル」

Conductor は 7B の管理モデルとして、タスクごとに「誰を呼ぶか」「何を依頼するか」「どの文脈を渡すか」を学習。簡単な問題は 1 モデル、難しいコード問題は planner → coder → verifier のように動的なチームを組む。Fugu Max は最も軽い解決可能モデルへ動的ルーティングし、コスト性能の Pareto frontier を押し広げる。

モデル選択

分業設計

検証



タスク難度に応じて
構成自体を変える

両者を同じ座標で見ると

観点	従来 LLM	Jev	Sakana AI
主役	回答を生成	判断を返す	モデル群を編成
出力	token 列	確率付き decision	workflow / routing
性能向上	本体を強化	判断コストを圧縮	既存知能を組合せ
向く場所	最終回答	細かい分岐・制御	複雑タスク・agent 運用

Sources: TypeSafe AI (Jev / RLCD / parallel sampler); Sakana AI (Conductor, Fugu Max, Transformer²). 数値・表現は各社公式発表に基づく要約。

2. OpenAI の好条件と、単一フロンティアモデル中心戦略の弱点

ここからは公開情報を踏まえた「構造上の可能性」として整理する。

OpenAI：モデル階層そのものがルーティング資産

GPT-5.6 は、Sol（旗艦）・Terra（バランス）・Luna（最速・低コスト）を明示的に役割分け。さらに reasoning effort も複数段階を持つ。

この「性能・速度・価格の階層」が、Jev 的 분류器や Sakana 的 router の選択肢になる。



↓ もし入力ごとに「必要な能力」を高精度分類できれば

簡単 → Luna | 通常 → Terra | 難しい → Sol | 極難 → Astra + verifier / tools

「全件を最上位モデルで処理」せず、品質を保ったまま平均 compute を下げる余地がある。

Anthropic：Mythos/Fable 級を頂点に置く「強い単体モデル中心」の場合

Anthropic 自身は Haiku / Sonnet / Opus など複数階層を持つため「一点集中」と断定はできない。一方、Fable 5.1 と Mythos 5.1 は同じ基盤モデルで、

Mythos は高能力領域向けにアクセスを絞る設計。最上位モデルの能力向上を中心に性能を押し上げる戦略には、構造上の弱点もある。

平均コストが高止まり

簡単な仕事にも大きなモデルを使うと、性能は高くても価格性能比が伸びにくい。

低遅延の分岐が弱い

「ここだけ深く考える」「ここは軽量経路」の粒度が粗いと、全体最適化が難しい。

改善がモデル更新待ち

単体性能の伸びが鈍化すると、システム全体の性能向上も連動して鈍化しやすい。

結論：次のジャンプは「モデル更新」ではなく「perfect routing」から来る可能性

OpenAI は、役割の違うモデル群と推論強度を既に持っているため、Jev のような高速分類器と Sakana AI のような学習済みルータを吸収したときの伸びしろが大きい。

単体モデルの“賢さ”が一時的に伸び悩んでも、判断・routing・verification・compute 配分の最適化だけで、Fable 登場時のような価格性能比の段差が再び起こり得る。